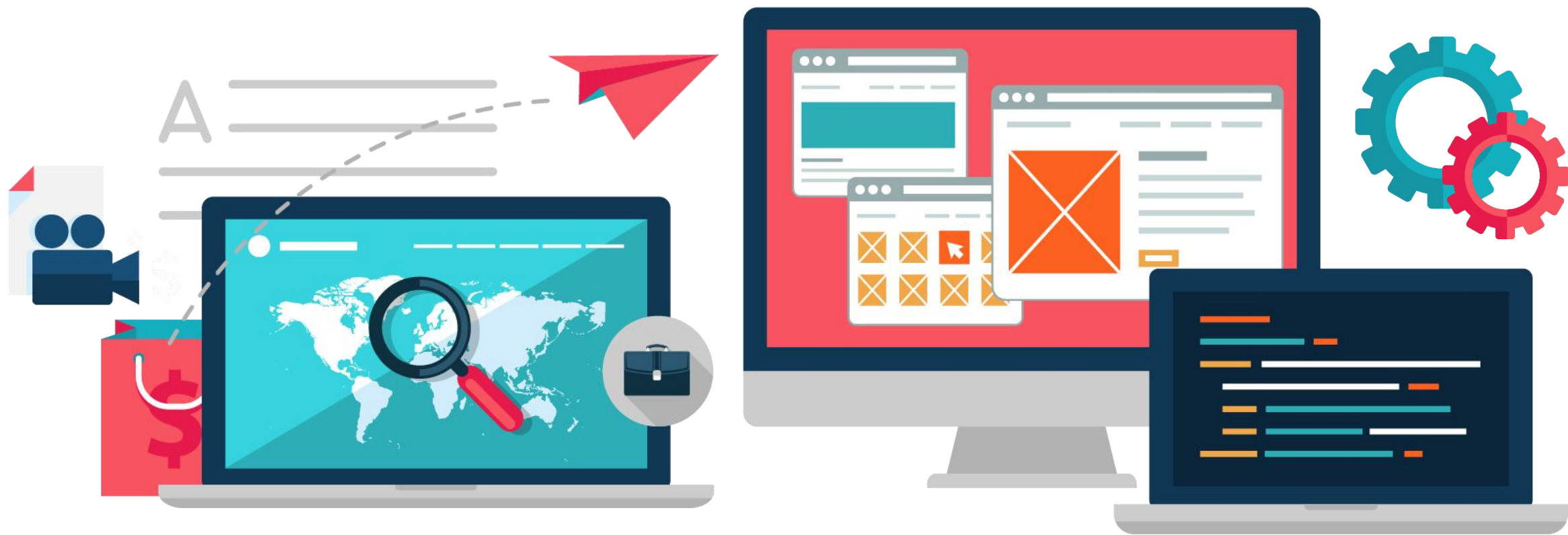


面向对象程序设计

Object Oriented Programming





类的设计

01

面向对象的三大基本特征

02

抽象与封装

03

信息隐蔽

04

分离使用 and 实现

05

类的设计

06

类的设计(例)





抽象(OOA):

通过对问题域中事物的分析, 识别本质特征, 总结和提炼事物的属性和行为

产出: 分析类(概念模型), 回答 “系统应该做什么”



封装和信息隐蔽(OOD):

将数据(属性)与操作(行为)捆绑成一个整体(类), 并隐藏内部实现细节

对外只暴露必要的公共接口, 内部细节不可见(解耦)

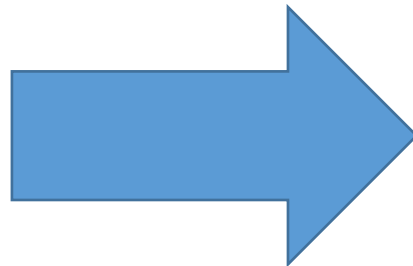
信息隐蔽是设计原则和目标, 封装是实现机制和方法

产出: 设计类(可实现的类蓝图), 回答 “系统如何做”



◆抽象(分析)

- ◆类型的抽象
- ◆行为的抽象
- ◆数据的抽象



◆表示(设计与实现)

- ◆类型的表示
- ◆行为的表示
- ◆数据的表示



例1：小球碰撞问题描述

- 类型的抽象： 小球， 边界墙(框)

- 行为的抽象：

小球： 移动、 设置速度、 设置位置、 检测是否碰撞、 反弹、 设置能量系数

边界墙： 设置位置、 改变能量系数、 检测是否碰撞、 反弹

- 数据的抽象： 位置、 速度、 系数移、 速度、 系数



■ 类型的表示

```
class Ball { };    class Border { };
```

■ 行为的表示

```
class Ball {  
    setPosition( );  
    setSpeed( );  
    stepMove( );  
    setScale( );  
    rebound( );  
};
```

```
class Border {  
    setPosition( );  
    setScale( );  
    rebound( );  
};
```

■ 数据的表示

```
float pos;    float radius;  
float scale;  float speed;
```



抽象及表示(设计与实现)

```
class Ball { // 设计结果
public:
    void setPosition(int x,int y);
    void setSpeed(int speed);
    void stepMove( );
    ...
private:
    int mx; int my;
    int mspeed;
    float mscale;
    Border * mborder;
};
```

```
void Ball::stepMove( ) { // 实现
    if ( HitBorder( ) ) {
        SetSpeed(...);
        mx = ...; my=...;
    } else {
        mx = ...; my=...;
    }

    setPosition(mx,my);
}
```

```
class Border {
public:
    void setPosition(int x,int y);
    void setScale( );

    ...
private:
    int mx; int my;
    float mscale;
};
```

例2：一个游戏中有多种怪物(Monster)，怪物之间可能要发生战斗(fight)，每场战斗都是一个怪物与另一怪物之间的一对一战斗。每个怪物都有自己的生命值(hitpoint)、攻击力值(damage)和防御力值(defense)。战斗时，两个怪物依次攻击对方，即怪物a首先攻击怪物b，然后轮到怪物b攻击怪物a，之后，怪物a再次攻击怪物b，...，直到一方生命值为0。每种怪物都有各自特有的攻击(attack)方式，产生相应的攻击效果。请根据你对上述描述的理解，定义并实现怪物类Monster。

抽象

类型的抽象：怪物类

行为的抽象：战斗、setter/getter，从调用者角度考虑

数据的抽象：生命值、攻击力值、防御力值

```
class Monster {  
public:  
    ....  
    bool Fight( Monster & other )  
        {   }  
};
```

```
bool Monster::Fight( Monster & other ) {  
    while(1) {  
        if ( Attack(other) )      return WIN;  
        if (other.Attack(*this))  return LOSE;  
    }  
}
```



■ 有意义的事物都可以抽象

只要在问题域中具有明确边界的事物，无论具体或抽象、虚拟或现实
例：汽车、权限、网络请求、窗口、计费规则、三体世界、魔法咒语

■ 通过封装进行表示(设计与实现)优化

例：

```
void Login(const char * username, // 散乱，难维护
           const char * password,
           const char * validCode,
           DateTime    dt,
           int          clientID,
           const char * IP );
```

```
void Login( LoginInfo& info); // 参数封装，提高内聚，简化接口
```

```
void Login( UserInfo& user,
           Address& client); // 封装时按职责分离
```



■ 隐藏行为、操作：

事物的一个行为特征可由多个操作过程构成，在C++中一个操作通常由一个或多个成员函数实现。

设计时，可隐藏全部或部分成员函数

■ 隐藏数据：

面向对象程序设计在抽象时**更关注事物的行为特征**，在表示时把数据隐藏起来更能体现事物的本质特征。



```
class Ball {  
public:  
    void SetPosition (int x,int y) ;  
    void SetSpeed (int speed) ;  
    void StepMove () ;  
private:  
    bool HitBorder( );  
private:  
    int mx; int my;  
    int mspeed;  
    float mscale;  
    Border * mborder;  
};
```

```
void Ball::StepMove( ) {  
    if ( HitBorder( ) ) {  
        SetSpeed(...);  
        mx = ...; my=...;  
    } else {  
        mx = ...; my=...;  
    }  
  
    SetPosition(mx,my);  
}
```

```
bool Ball::HitBorder( )  
{  
    bool hitLeft  = distance<=radius;  
    bool hitRight = (distance+radius>=RIGHT);  
  
    return (hitLeft || hitRight);  
};
```



- 减少外部可见行为和数据
- 对象间只通过可见行为交互
- 更安全、可靠
- 维护方便
- 便于复用



- 类的使用者---调用方
- 类的实现---成员函数实现、数据的表示、组织等

```
class Client {  
public:  
    void useImp(Imp & imp) {  
        imp.pubFunc( );  
        imp.priFunc( ); //错误  
        imp.fields = 100; //错误  
    }  
};
```

```
class Imp {  
public:  
    void pubFunc( ) { ... }  
private:  
    void priFunc( ) { ... }  
private:  
    int fields;  
};
```



建议:

- 尽量减小使用和实现间的耦合度
接口应越少越小, 如人和洗衣机
- 前置声明好于include
- 定义类时尽量减小与运行环境、平台、硬件系统等关联

类B的使用如

```
class A {  
public:  
    void F( B& b) {  
        b.Func( );  
        .....  
    }  
};
```

类B的实现

```
void B::Func( ) { /* 实现 */ }
```



分离使用和实现 (对比)

```
//student.h
#include <iostream>
class Student{
public:
    void input() {
        int a;
        std::cout<< "请输入年龄:"
        std::cin>>a;
        age = a;
    }
    void f( );
private:
    int age;
};
```

```
//student.h
class Student{
public:
    Student(int theAge) { }
    void f( );
private:
    int age;
};
```

```
//MainApp.cpp
#include <iostream>
#include "student.h"
int main() {
    int a;
    std::cout<< "请输入年龄:"
    std::cin>>a;
    Student stu(a);
    stu.f( );
}
```



■ 多个类的设计

- ✓ 类的拆分
- ✓ 类的合并
- ✓ 数据关系
- ✓ 行为关系

应该拆分类的情况：

1. 类有多个明显不同的职责
2. 类太大难以维护和理解
3. 类的某些方法只使用部分成员变量(内聚性低)
4. 类的某些功能可能需要在其他上下文中重用

应该合并类的情况：

1. 多个类总是同时使用
2. 类之间的耦合度过高
3. 类过于简单，没有独立存在的必要
4. 继承层次过深，可以扁平化

■ 多个类的设计

- ✓ 类的拆分
- ✓ 类的合并
- ✓ 数据关系
- ✓ 行为关系

```
//application.h
#include<iostream>
class Application{
public:
    Application() ;
    void Init( );
    void Run(Argument & arg);
    void Term( );
private:
    Input * in;
    Output * out;
    Config * cfg;
};
```

```
class Argument
{ };
```

```
class Input
{ };
```

```
class Output
{ };
```

```
class Config
{ };
```

明确区分类之间的数据关系是依赖、关联、聚合还是组合，它们代表不同的语义。

■ 多个类的设计

- ✓ 类的拆分
- ✓ 类的合并
- ✓ 数据关系
- ✓ 行为关系

```
class A{  
public:  
    void fA(B & b,C& c) {  
        .....  
    }  
};
```

```
class B {  
public:  
    void fB(A& a,C&c );  
};
```

```
class C {  
public:  
    void fC( A&a,B& b);  
};
```

```
class A{  
public:  
    void fA(B & b,C& c) {  
        b.fB( *this,c);  
        c.fC( *this,b);  
    }  
};
```

涉及多个类的行为关系可抽象分解在不同类中进行表示，并优先使一个类成为主控类，作为功能的主要实现者或行为的协调者。

多个类之间通过成员函数的参数相互调用，即双向行为依赖，形成一种网状交互关系。实际开发中需要谨慎使用或尽量避免，因为可能导致高耦合。



■ 单个类的设计

- ✓ 行为的可见性
- ✓ 行为参数的封装
- ✓ 依赖与关联
- ✓ 实现的封装
- ✓ 数据的封装
- ✓ 类方法与实例方法
- ✓ ...

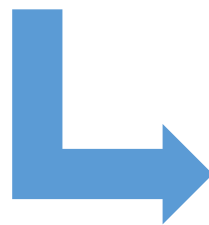
```
//application.h
#include <iostream>
class Application{
public:
    Application() ;
    Init( );
    Run(Argument & arg);
    Term();
private:
    bool loadFile(string&);
    bool saveFile(string&);
private:
    Input    *    in;
    Output  *    out;
    Config   *    cfg;
};
```



■ 单个类的设计

- ✓ 行为的可见性
- ✓ 行为参数的封装
- ✓ 依赖与关联
- ✓ 实现的封装
- ✓ 数据的封装
- ✓ 类方法与实例方法
- ✓ ...

```
class Dog{  
public:  
    void Bark(Human& human);  
    void Bark(Humen & humen);  
    void Bark(Door& door);  
    void Bark(Door& door, int db);  
    void Bark( int db);  
  
    ...  
};
```



```
class Dog{  
public:  
    void Bark(Dest & aDest);  
  
    ...  
};
```



■ 单个类的设计

- ✓ 行为的可见性
- ✓ 行为参数的封装
- ✓ 依赖与关联
- ✓ 实现的封装
- ✓ 数据的封装
- ✓ 类方法与实例方法
- ✓ ...

```
class Dog{  
public:  
    void Bark(Dest & aDest);  
    ...  
};
```



```
class Dog{  
public:  
    void setDest(Dest *);  
    void Bark();  
    ...  
private:  
    Dest * aDest  
};
```



■ 单个类的设计

- ✓ 行为的可见性
- ✓ 行为参数的封装
- ✓ 依赖与关联
- ✓ 实现的封装
- ✓ 数据的封装
- ✓ 类方法与实例方法
- ✓ ...

```
class Dog{  
public:  
    void func() {  
        int x=....;  
        int y=cac1(x);  
        do1();  
        ....  
};
```

甚至可以将整个函数实现全部委托给另一个对象，也就是通常说的委托(delegate).

```
void func() {  
    pObj->somefunc();  
}
```

```
class Dog{  
public:  
    void Func( MyObj & obj)  
{  
        obj.func1();  
        do1();  
        obj.func2();  
}  
};
```

```
class Dog{  
public:  
    void Func() {  
        pObj->func1();  
        do1();  
        pObj->func2();  
    }  
private:  
    MyObj * pObj;  
};
```



■ 单个类的设计

- ✓ 行为的可见性
- ✓ 行为参数的封装
- ✓ 依赖与关联
- ✓ 实现的封装
- ✓ 数据的封装
- ✓ 类方法与实例方法
- ✓ ...

```
class Dog{  
public:  
    void Func( ) {  
        age = 8;  
        ...  
    }  
private:  
    int age;  
    int color;  
    History & hist;  
    ...  
};
```

```
class Dog{  
public:  
    void Func( ) {  
        data->SetAge(8);  
    }  
private:  
    DogData * data;  
};
```

```
class DogData{  
public:  
    ....  
    void SetAge(int a)  
    { age =a; }  
private:  
    int age;  
    int color;  
    History & hist;  
    ...  
};
```



```
class Balloon {  
public:  
    Balloon( ) { }           //普通气球  
    Balloon (int color) { }  //有颜色气球  
    Balloon (Gas & gas) { } //充气气球  
    //...  
};  
Balloon normalBalloon;  
Balloon redBalloon(0xFF0000);  
Balloon heliumBalloon(heliumGas);
```

```
class File {  
public:  
    // 从磁盘创建文件  
    File(const string& filename);  
    File(const char* data, size_t size);  
    // 从内存创建文件  
};  
  
File diskFile("data.txt");  
File memoryFile(buffer, size);
```

重载构造函数的主要优势：

1. **语法简洁性**，创建对象时直接调用构造函数，代码更短更直观；
2. **语言原生支持**，无需额外封装，符合C++核心语言规范；
3. **性能最优**，无额外开销，适合高性能场景(如数学运算类)；



创建方法、静态工厂方法或命名构造函数惯用法

```
class Balloon {  
public: static Balloon CreateNormalBalloon() { return Balloon (); }  
        static Balloon CreateColorBalloon( int c ) { return Balloon(c); }  
        static Balloon CreateGasBalloon(Gas& g) { return Balloon(g); }  
private: Balloon( ) { }  
         Balloon (int color) { }  
         Balloon ( Gas & gas) { }  
};
```

```
Balloon normal = Balloon::CreateNormalBalloon();  
Balloon red = Balloon::CreateColorBalloon(0xFF0000);  
Balloon flying = Balloon::CreateGasBalloon(helium);
```

```
class File {  
public: static File OpenFromDisk(const std::string& filename);  
        static File CreateFromMemory(const char* data, size_t size);  
private: File(const std::string& path, bool isDiskFile);  
         File(const char* data, size_t size, bool isDiskFile);  
};
```

```
File diskFile = File::OpenFromDisk("data.txt");  
File memoryFile = File::CreateFromMemory(data, 11);
```



- 静态工厂方法的主要优势：
- 1. **语义明确性**，通过方法名直接表达意图，避免重载构造函数的歧义；
 - 2. **构造过程控制**，封装复杂逻辑隐藏实现细节，如校验、单例、缓存等；
 - 3. **多态构造能力**，可返回派生类对象或代理对象；
 - 4. **适合中大型项目或库开发**，提供全局可访问的稳定的公共接口；

特性	重载构造函数	静态工厂方法
语义表达	依赖参数类型	通过方法名自文档化
构造控制	仅限简单初始化	可封装复杂逻辑
多态支持	不支持	可返回子类或代理对象
错误处理	只能抛异常	可返回nullptr或错误码等
ABI稳定性	修改易破坏	内部变化不影响接口
性能	零开销	可能有调用层开销



例：单件的实现

```
class Ball {  
public:  
    static Ball & GetInstance () {  
        static Ball aBall;  
        return aBall;  
    }  
private:  
    //私有化构造函数，防止外部访问，  
    //或者 编译器提供缺省的构造函数  
    Ball() {}  
    //同理  
    Ball (const Ball&) ; //只声明，不实现，或delete  
    Ball& operator=(const Ball& ); //只声明，不实现  
public:  
    void Func( );  
};
```

- class Ball { };
- 希望使用者至多只能创建Ball的一个实例

```
class Ball {  
public:  
    static Ball * GetInstance () {  
        if (pBall == nullptr)    pBall = new Ball;  
        return pBall;  
    }  
    static void ReleaseInstance()  
        { delete pBall;    pBall=nullptr;}  
private:  
    static Ball *pBall; //不要忘记类外分配存储空间  
    Ball() { }  
    Ball (const Ball&) ; //只声明, 不实现, 或删除  
    Ball& operator=(const Ball& ); //只声明, 不实现  
public:  
    void Func( );  
};
```

- class Ball { };
- 希望使用者至多只能创建Ball的一个实例



```
class Paginate {  
};
```

« 1 ... 6 7 8 9 10 ... 22 » Go

« 1 ... 6 7 8 9 10 ... 22 » Go

« 1 ... 6 7 8 9 10 ... 22 » Go

« 1 2 3 4 5 ... 22 » Go

« 1 ... 16 17 18 19 20 ... 22 » Go

« 1 ... 18 19 20 21 22 » Go



```
class Paginate {  
public:  
    void go(int page);  
    void prev( );  
    void next( );  
    void prevN( );  
    void nextN( );  
};
```

```
class Paginate {  
private:  
    int num;  
    int total;  
    int rangeStart;  
    int rangeEnd;  
};
```

其它：

构造函数-需要自定义

拷贝构造函数-不需要自定义

赋值函数-不需要自定义

getter函数或show()-具体看需要



本章结束